Ders 5 Genişletilmiş Notu: Kelime Yörüngeleri, word2vec ve FastText, Aktif ve Yarı Eğiticili Öğrenme

Ders 5 Genişletilmiş Notu: Kelime Yörüngeleri, word2vec ve FastText, Aktif ve Yarı Eğiticili Öğrenme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kelime Koordinatlarından Metin Koordinatına: Ortalama Alma
- 2. Anlamsal Uzayların Metin Sınıflandırmada Kullanımı
- 3. Çok Anlamlı Kelimeler Sorunu
- 4. Kelime Yörüngeleri
- 5. Sinirsel Dil Modellerine Giriş: word2vec
- 6. word2vec'in Hata Fonksiyonu ve Negatif Örnekleme
- 7. FastText: Kelime İçi n-gram Benzerliği
- 8. Hazır ve Kendi Eğitilmiş Vektörlerle Benzer Kelimeler
- 9. Kelime Vektörlerini Değerlendirmek
- 10. Cümle ve Metin Vektörleri
- 11. Kelime Öbekleri
- 12. Bölüm 4'e Giriş: Makine Öğrenmesinin İkinci Versiyonu
- 13. Aktif Öğrenme
- 14. Yarı Eğiticili Öğrenme
- Örnek Sorular
- Dersin Toparlaması

1. Kelime Koordinatlarından Metin Koordinatına: Ortalama Alma

Geçen ders kelimelere vektör atama işleminden ve bunun için kullanılabilecek birçok yöntemden söz edilmişti. En son yöntem birlikte geçme sayılarına dayanıyordu. "Birlikte geçme" farklı şekillerde tanımlanabilir: aynı cümlede, aynı paragrafta ya da aynı metinde birlikte geçmek gibi. Bütün kelimelerin bütün kelimelerle birlikte geçmelerinden bir matris oluşturulur, benzerlikten uzaklığa geçilir ve bu matris, Principal Component Analysis'e benzeyen Multidimensional Scaling (MDS) yöntemine verilerek kelimelerin bu uzaklıklara uyumlu koordinatları bulunur.

Sonraki soru şuydu: kelimeler için bulduğumuz koordinatları daha büyük metinler, örneğin bir cümle ya da bir dokümanın tamamı için de elde edebilir miyiz? Akla gelen en basit yaklaşım ortalama almaktır.

Hoca bunu ekrana yazdığı bir örnekle anlattı (ekrandan, 00:10:00):

```
Ali   = [0.7  -0.2 ...... 0.8]   300d
Yemek = [0.5   0.9 ...... 0.7]   300d
Farklı = [...]
Geldi  = [...]

"Ali yemek" = [0.6  0.35 ... 0.75]   300d
```

Kelimelerin hepsi 300 boyutlu ve aynı uzaydadır. Bu, birinci boyutun bütün kelimelerde aynı şeyi, ikinci boyutun bütün kelimelerde aynı şeyi ifade ettiği anlamına gelir. "Ali yemek" diye bir metnin koordinatını bulmak için iki kelimenin birinci boyutlarının ortalaması metnin birinci boyutu, ikinci boyutlarının ortalaması metnin ikinci boyutu olarak alınır:

- Birinci boyut: $(0.7 + 0.5)/2 = 0.6$
- İkinci boyut: $(-0.2 + 0.9)/2 = 0.35$
- Son boyut: $(0.8 + 0.7)/2 = 0.75$

Metinde 3, 5 ya da 10 kelime olsaydı yine bütün kelimelerin her boyuttaki ortalaması alınacaktı; sonuç yine 300 boyutlu olacaktı.

Hoca burada sınıfa "yemek Ali"nin değerlerinin ne olacağını sordu. Cevap: aynı. Çünkü ortalama almak sıralamaya bakmaz. Düzenli bir cümle de, onun karmakarışık hali de aynı vektörü verir. Hoca bunun yine de işe yaradığını vurguladı: kelimelerin sırası karışık olsa bile metnin neden bahsettiğine dair kaba bir görüş elde ederiz. Ortalamadan başka işlemler de yapılabilir, ama en basiti budur.

Ortalamanın güzel tarafı şudur: iki metin farklı sayıda kelime içerse bile ikisi de kelimelerle aynı boyutlu (burada 300 boyutlu) aynı uzaya dönüşür. "Farklı yemek" gibi başka bir metin de 300 boyutlu olur ve diğerleriyle karşılaştırılabilir.

2. Anlamsal Uzayların Metin Sınıflandırmada Kullanımı

Geçen derste görülen haber sınıflandırma çalışmasına geri dönüldü. Beş farklı haber türü vardı. Her haberin koordinatı, içindeki kelimelerin koordinatlarının ortalamasıyla bulunmuş; birinci ve ikinci boyutlar çizdirilip haberler türlerine göre renklendirildiğinde fena olmayan bir ayrışma görülmüştü.

Bu derste sınıflandırma başarıları gösterildi (ekrandan, 00:12:45). Sözlükte 4500 kelime vardır. Haberler bu yöntemle 100 ve 10 boyuta indirgenmiştir:

Sınıflandırma algoritması	100 boyutlu metinler	10 boyutlu metinler
Lineer regresyonla sınıflandırma	93.25	89
Pace regresyonla sınıflandırma	92.75	88.5
En yakın komşu	70	79
Destek vektör makineleri	90	89.75
Rastgele orman (100 ağaçlı)	90.75	87.75

Yandaki tabloda aynı veri 4500 boyutlu terim-doküman temsiliyle sınıflandırılmıştı. Örneğin tf ile klasik Naive Bayes %87.25, ayrık (diskrit) Naive Bayes %85.75 (tf-idf ile %89.25), destek vektör makineleri %87, en yakın komşu %34.25 başarı vermişti. Rastgele orman ve regresyonla sınıflandırma satırlarında 4500 boyut için sonuç yerine "X" yazılıydı.

Hoca, normalde bir dokümanı 4500 boyutta, çok seyrek (sparse) bir temsille ifade etmek gerekirken burada 100 ya da 10 boyutlu, çok daha yoğun bir temsile çok fazla bilgi kaybetmeden inildiğini vurguladı. Konu sınıflandırma bazında %90 civarına çıkan başarılar elde edilebilmiştir. Oldukça basit bir tekniktir: daha yüksek başarı ve metin temsilinde daha az hafıza.

Dezavantajları da vardır:

- **Matrisi oluşturmak:** 4500 kelime varsa 4500×4500'lük bir birlikte geçme matrisi elde etmek gerekir. Sözlük boyutu çarpı sözlük boyutu bir yapıdır ve sözlük büyüdükçe bu hızla artar.
- **Matristen koordinata geçmek:** Bu matris üzerinde MDS gibi çarpanlara ayırma işlemleri uzun sürer. İki süreç de pek ölçeklenebilir değildir.

Birlikte geçmeleri bulurken Google'ın neden kullanılmadığı da bu yüzdendir: hocanın verdiği tahminle 16 milyon sorgu göndermek çok uzun sürerdi.

> **Ders sonrası not:** 4500 kelimenin farklı ikili sayısı $4500 \cdot 4499/2 \approx 10.1$ milyondur. Hocanın söylediği 16 milyon büyük olasılıkla kaba bir tahmindir ya da sorgu düzeninin farklı olmasından kaynaklanır; asıl mesaj, sorgu sayısının kelime sayısının karesiyle büyümesidir.

Yöntem dilden bağımsızdır: A dili ile B dili arasında bir fark yoktur. Yalnız birlikte geçmeyi tanımlarken o dile özel bir şey gerekiyorsa o değişebilir.

3. Çok Anlamlı Kelimeler Sorunu

Eş sesli, yani birden fazla anlamı olan kelimelerin koordinatına çok güvenemeyiz. Hoca bunu "Jaguar" örneğiyle anlattı. Bir tarafta Jaguar, Ferrari, Lamborghini gibi araba markaları; diğer tarafta Jaguar, çita, kaplan gibi hayvanlar vardır. Jaguar'ın koordinatı bir yandan Ferrari ve Lamborghini'ye, bir yandan da çita ve kaplana benzemeye çalışır. Hem bunlara hem ötekilere benzeyebilmesi için bu iki grubun da biraz birbirine benzer hale gelmesi gerekir. Bu istenmeyen bir durumdur: benzer olmayan şeyler benzer hale gelir.

Örneğin "Ferrari" ile "çita" hiçbir yerde birlikte geçmez, ama ikisi de "Jaguar" ile birlikte geçtiği için yaklaşırlar. Hoca buna ikinci dereceden birlikte geçme dedi; bu durumda işimize yarayan bir sonuç çıkmaz.

Çözüm olarak Jaguar'a birden fazla vektör verilebilir. Ancak o zaman metindeki bir Jaguar'ın hangi anlamda kullanıldığına dair ek işlemler gerekir; bu da kolay ve ölçeklenebilir bir çözüm değildir.

4. Kelime Yörüngeleri

4.1 Fikir

Hoca, kendi grubunun zamanında yaptığı bir çalışmayı anlattı. Bu kez kelimelerin koordinatları değil, metinlerin kendisi bir **yörünge** olarak temsil edilebilir mi diye düşünülmüştü. Her kelimenin bir koordinatı olduğunu varsayalım. "Ali okula giderken yolda onu gördü" cümlesinde Ali'den başlanır, okula gidilir, okuldan "giderken"e, oradan "yolda"ya, "onu"ya ve en son "gördü"ye gidilir. Kelime koordinatları sırayla birleştirilince yönlü bir yol, bir yörünge ortaya çıkar.

Önemli nokta, bu yörüngenin kelimelerin koordinatlarından bağımsız özelliklerinin kullanılmasıdır. Varsayım şudur: bu özellikler metni yazan kişiye dair ipuçları verir. Bir kişi A konusunda da, hiç ortak yanı olmayan B konusunda da yazabilir; ama A'yı yazarkenki yörüngesi ile B'yi yazarkenki yörüngesi arasında ortak özellikler vardır. Hoca bu iddiaya karşı çıkılabileceğini de belirtti; bununla ilgili çeşitli deneyler yapılmış ve fena olmayan sonuçlar alınmıştı.

Kullanılabilecek özelliklere örnek olarak açılar verildi: ardışık üç kelimenin oluşturduğu açı. Kelimeler iki boyutta olmak zorunda değildir, çok daha yüksek boyutta olabilirler; orada da açı tanımlamak mümkündür. Bir kelime atlanarak da açı ölçülebilir: "okula giderken yolda" yerine "okula, yolda, onu" üçlüsü gibi. Böylece kelimelerin bireysel koordinatları yerine yörüngelerden bilgi çıkarılır.

4.2 Araştırma soruları ve örnek

Çalışmanın araştırma soruları şunlardı: Aynı kişinin farklı yazılarının yörüngeleri birbirine benzer mi? Farklı kişilerin yörüngelerini birbirinden ayırmak mümkün mü? Bu sorulara cevap vermek için önce yörünge oluşturmak, bunun için de kelime koordinatlarını bulmak gerekir.

Slaytta iki örnek metin vardı (ekrandan, 00:26:55):

- "Bugün yolda kedi gördüm. Arkasından gittim. Hızlı koşuyordu. Yakalayamadım."
- "Hızlı araba kullanmak tehlikelidir. Kediler arabaların altında kalabilirler. Bugün neredeyse eziliyordu bir kedi."

İki metin aynı uzayda ("kedi", "hızlı", "araba", "yol", "bugün", "yakala", "koş" gibi kavramların yer aldığı bir uzayda) birer yörünge olarak çizilmişti.

4.3 Yörünge özellikleri

İki tür özellik kullanılmıştır:

- **Kavramlar arası mesafe:** Örneğin ardışık kavramlar arasındaki ortalama mesafe. Bu, metnin "oradan oraya zıplayıp zıplamadığını" kabaca gösterir. Bağlamdan bağlama zıplamak, ardışık kavramlar arası mesafenin yüksek olması demektir.
- **Kavramlar arası açı:** Ardışık üç kavramın oluşturduğu açı.

Mesafeler $(n, n+1)$, $(n, n+2)$, $(n, n+3)$ çiftleri için ölçülür. Burada bir sorun vardır: bir metin 5, bir başkası 15 kelimeden oluşur. Hangi açılar ve hangi mesafeler kullanılacak? Amacımız örnekleri aynı uzayda temsil etmektir; birini 10 boyutlu, birini 20 boyutlu uzayda temsil edersek aynı uzayda olmazlar. Uzun ve kısa metinlerin temsil boyutunu eşitlemek gerekir.

Çözüm histogram kullanmaktır. Slayttaki tanım (ekrandan, 00:30:00):

> d uzunluğundaki bir yörüngede $d-1$ adet $(n, n+1)$ arası mesafe ölçülmektedir. Bu $d-1$ ölçümün eşit aralıklı 10 parçalık histogramı çıkarılmaktadır. Bu histogramın 10 adet değeri ve her değerin bir frekansı bulunmaktadır. Dolayısıyla yörüngenin $(n, n+1)$ arası mesafelerini ifade eden 20 adet özellik çıkarılmaktadır. Bu işlem $(n, n+2)$, $(n, n+3)$ arası mesafelere ve $(n, n+1, n+2)$, $(n, n+2, n+3)$ arası açılara da uygulandığında bir metni ifade eden yörüngeye ait 100 özellik bulunmuş olmaktadır.

Yani beş histogram (üç mesafe, iki açı) × 20 özellik = 100 özellik. Kelime sayısı ne olursa olsun histogram alındığı için boyut sabit kalır. Histogram, normalizasyon yapılırsa "aralarındaki mesafesi şu olanların yoğunluğu bu kadar" ya da "aralarındaki açısı şu olanların yoğunluğu şu kadar" bilgisini verir.

4.4 Yazar tanıma deneyleri

Uygulama yazar tanımadır: bir yazı verildiğinde yazarı kimdir? 10'lu çapraz geçerleme kullanılmıştır (ekrandan, 00:32:40).

İki yazara ait 35'er yazı (toplam 70 metin):

Algoritma	Sınıflandırma başarısı (%)
C4.5	97.15
Naive Bayes	97.15
En yakın komşu	95.71
Destek vektör makineleri	98.57

Her yazara ait 35'er yazı, daha fazla yazar:

Algoritma	3 yazar	4 yazar	8 yazar
C4.5	91.43	90.71	53.21
Naive Bayes	95.24	92.14	62.5
En yakın komşu	80.95	62.14	50
Destek vektör makineleri	94.29	85.71	68.57
Zero Rule	33.33	25	12.5

Yazar sayısı arttıkça iş zorlaşır. Hoca, kullanılan bilginin aslında çok az olduğunu hatırlattı: yalnız 100 özellik vardır, kelimelerin koordinatlarına, yani içeriğe hiç bakılmaz. Bu düşünülünce 8 yazarı birbirinden ayırt etmek için bu sonuç hiç fena değildir. Yörüngenin güzel tarafı, sınıfların aynı konuda olmasının gerekmemesidir: aynı yazar hem ekonomi hem spor yazabilir, ama yörünge özelliklerinin kişiye özel olduğu tahmin edilir.

> **Ders sonrası not:** Hoca konuşurken 3 yazar için "%93", 8 yazar için bir yerde "%78" dedi; slayttaki tabloya göre en iyi sonuçlar 3 yazarda %95.24 (Naive Bayes), 8 yazarda %68.57'dir (destek vektör makineleri). Hoca da hemen ardından %68'i andı.

Bir de şu soru tartışıldı: koordinatlar arasındaki farklar bir kavrama karşılık gelebilir mi? "Hızlı" ile "yakala" arasındaki fark vektörünü kullanıyoruz. Bunun normalde bir anlamı yok gibi görünür, ama olabilir; fark vektörünün bir anlamsal karşılığı olabilir ve yöntem farklara baktığı için bunu kullanıyor olabilir.

4.5 Olası kullanım alanları

- Psikolojik hastalıkların tespiti: psikolojik hastalıklar düşünce süreçlerini, yani bu yörüngeleri nasıl etkiliyor?
- Cinsiyet, yaş ve eğitim farklılıklarının düşünce süreçlerine etkisi. Örneğin "kadınlar daha fazla daldan dala atlar, erkekler daha düz düşünür" gibi klişe ifadeler gerçekten doğru mu?

Hocanın grubu zamanında kadın ve erkeklerin yörüngelerini ayırt etmeyi denemişti. Sonuç olumsuzdu: iki yazarı birbirinden ayırt etmek mümkündü, ama erkekleri bir sınıfa, kadınları bir sınıfa koyduğunuzda yörünge farklılıkları çıkmıyordu. Kişisel farklar bulunabiliyor, ama cinsiyet bazında bir ayrım ortaya çıkmamıştı.

5. Sinirsel Dil Modellerine Giriş: word2vec

5.1 Yapı

Aradan sonra sinirsel dil modellerine geçildi (ekrandan, 00:53:10). İlk model word2vec'tir (Mikolov ve ark., 2013, "Distributed representations of words and phrases and their compositionality"). word2vec çok basit, tek gizli katmanlı bir yapay sinir ağıdır: bir giriş katmanı, bir gizli katman (projection layer) ve bir çıkış katmanı vardır.

```
Skip-gram modeli

  Input layer            Projection layer          Output layer
  1-hot input vector     embedding for w_t         probabilities of context words
  (1 x |V|)    --- W (|V| x d) --->  (1 x d)  --- C (d x |V|) --->  (1 x |V|)
      w_t                                                            w_{t+1}
```

- $|V|$: sözlük boyutu (vocabulary size).
- d: kelimelerin kaç boyutlu uzayda temsil edileceği (örneğin 100 ya da 300).
- İki ağırlık matrisi vardır: W ($|V| \times d$) ve C ($d \times |V|$). Hoca bunların birbirinin transpozu olarak da kurulabildiğini, ama bunun bizim için önemli olmadığını söyledi.

5.2 Skip-gram ve C-bow

Her kelime için bir pencere tanımlanır: ortadaki kelimenin solundan ve sağından kaç kelime alınacağı. Bu sayıya C denir. Slayttaki tanımlar:

- $C = 2$ için bağlam $= (w_{t-2}, w_{t-1}, w_t, w_{t+1}, w_{t+2})$.
- **Skip-gram:** giriş w_t, çıkış $(w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$'nin her biri. Ortadaki kelime verilir, çevresindeki kelimeler tahmin edilir.
- **C-bow (CBOW):** giriş $(w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$'nin her biri, çıkış w_t. Çevre verilir, ortadaki tahmin edilir.

İki yapının temel mantığı aynıdır: bir taraftan bir kelime verilir, diğer taraftan bir kelime alınır. Verilen kelime ile alınan kelime arasındaki ilişki, aynı metin içinde yakın yerlerde geçmektir.

Bir kelimeyi ağa vermek, one-hot encoding ile olur. Sözlükte "kedi" kelimesinin bir indisi vardır; o indis 1, diğerleri 0 yapılır ve giriş olarak verilir. Çıkışta bir dağılım elde edilir; en yüksek değere sahip kelime çıktı olarak kabul edilir.

> **Ders sonrası not:** Hoca çıkışın "softmax'tan geçirilmeden doğrudan dağılım olarak" alındığını söyledi. Orijinal skip-gram modelinde çıkış büyük olasılıkla softmax ile olasılığa çevrilir; negatif örneklemeli eğitimde ise her (kelime, bağlam) ikilisi için sigmoid kullanılır (bkz. Bölüm 6).

5.3 Eğitim kümesinin oluşturulması

Eğitim kümesi bir derlemden üretilir (ekrandan, 01:10:00):

```
Text: w1 w2 w3 w4 w5 w6 w7 ...

Skip gram için eğitim örnekleri (giriş, çıkış), C=2 için:
  - benzer olmasını istediğimiz ikililer: (w3,w1), (w3,w2), (w3,w4), (w3,w5)
  - farklı olmasını istediğimiz ikililer: (w3,t1), (w3,t2), (w3,t3) ... (w3,tK)
    burada t1...tK p(w_i)'ye göre rasgele seçilen kelimeler
Sonra w4 ile devam ...
C-bow için ikilileri tersine çevir
```

w_3 verildiğinde w_1, w_2, w_4, w_5'in alınması istenir. Sonra w_4 için w_2, w_3, w_5, w_6 istenir ve pencere böyle kayar.

5.4 Eğitim neden benzer kelimeleri yaklaştırır?

w_3 verildiğinde çıkışta w_1, w_2, w_4 ya da w_5'ten herhangi birinin çıkması istenir. Hata fonksiyonu açısından bunların hangisinin çıktığı arasında büyük bir fark olmamalıdır. O halde aynı kelimenin çevresindeki kelimelerin birbirine yakın temsillere sahip olması gerekir. Eğitilen sistem giderek aynı kelimenin çevresindeki kelimeleri birbirine, hatta ortadaki kelimeyi de onlara yaklaştırır. w_3 adımında w_1, w_2 ve w_4, w_5 yaklaşır; w_4 adımında w_2, w_3 ve w_5, w_6 yaklaşır. Böylece birbirine yakın yerlerde geçen kelimeler, kaç boyutlu bir uzay seçildiyse (300, 500...) o uzayda sürekli birbirine yaklaştırılır.

5.5 Eğitimden sonra elde ne kalır?

Eğitim bittiğinde ağın iki ağırlık matrisi vardır. İlki $|V| \times d$ boyutludur: sözlük boyutu çarpı boyut. Bu matrisin her satırı, bir kelimenin yeni d boyutlu uzaydaki karşılığıdır. Yani sözlükteki her kelime için d boyutlu uzayda bir nokta oluşturulmuştur. Sistemi eğittikten sonra bu matris kelimelerin koordinatlarıdır; başka bir işlem yapılmaz.

> **Hocanın vurgusu:** Bu yöntemin çok güzel tarafı, hiçbir yerde $V \times V$'lik bir yapı olmamasıdır. Bir dili yeterince iyi temsil etmek için günümüzde 32 bin civarından başlayan büyük sözlükler gerekir. Önceki yöntemlerdeki $V \times V$ yapılar (örneğin 32 bin × 32 bin) işimizi çok bozar. word2vec'te yalnız $V \times d$, $1 \times d$ ve $d \times V$ boyutlu, çok daha küçük matrisler vardır.

6. word2vec'in Hata Fonksiyonu ve Negatif Örnekleme

6.1 Benzerlik ölçüsü olarak iç çarpım

W ve C başta rastgele atanır ve eğitim boyunca güncellenir. İstenen şudur: bir kelimenin w vektörü, çevresinde bulunan kelimelerin (c) vektörlerine benzer; kendisiyle aynı yerde geçmeyen kelimelerin vektörlerinden farklı olsun.

Kullanılan yapı: iki vektör benzerse iç çarpımları (dot product) yüksektir. İki vektör arasındaki açı darsa birbirlerine benzerler ve çarpımları yüksek çıkar.

> **Ders sonrası not:** Hoca "dot product bize açının tersini veriyordu" dedi. Doğrusu $a \cdot b = \|a\| \, \|b\| \cos\theta$'dır: iç çarpım açının kosinüsüyle orantılıdır, açı küçüldükçe büyür. Söylenmek istenen de büyük olasılıkla bu ters yönlü ilişkidir.

6.2 Skip-gram hata fonksiyonu

Hata fonksiyonunun ilk kısmı, kelime w ile bağlam kelimesi c'nin iç çarpımının sigmoidden geçirilmiş halinin logudur; w ile c'nin yakın olması istenir. Hoca logun hikâyede çok önemli bir anlamı olmadığını söyledi. İkinci kısım, w'nin uzak olması istenen kelimelerle ilgilidir.

> **Ek bilgi (derste anlatılmadı):** Mikolov ve ark. (2013) makalesindeki negatif örneklemeli skip-gram amacı büyük olasılıkla şu biçimdedir: $\log \sigma(c \cdot w) + \sum_{i=1}^{k} \mathbb{E}_{t_i \sim P(w)} \left[\log \sigma(-t_i \cdot w)\right]$. Burada σ sigmoid fonksiyonudur; ilk terim gerçek

bağlam ikilisinin iç çarpımını büyütür, toplam terimi $P(w)$'den seçilen k negatif kelimeyle iç çarpımı küçültür. Bu ifade en büyüklenir (hata olarak işaretinin tersi en küçüklenir).

6.3 Neden negatif örnek gerekir?

"w_3, w_1'e, w_2'ye, w_4'e, w_5'e benzer olsun" diyoruz. Peki w_3 neye benzer olmasın? Hoca bunun çok önemli olduğunu vurguladı.

> **Hocanın vurgusu:** Bir sisteme yalnız pozitif geri bildirim verirseniz ("şunlar birbirine benzesin" deyip hiçbir zaman "şunlar benzemesin" demezseniz) temsil uzayı bir noktaya çöker (collapse). Herkes aynı yere çöker, herkes herkese benzer ve sistem hiçbir şey öğrenmemiş olur.

Benzer olması gereken ikilileri metinden elde etmek kolaydır. Benzer olmaması gerekenler için Mikolov şunu yapmıştır: w ile birlikte, $P(w)$ dağılımından rastgele seçilen kelimeler kullanılır. Bu seçim düzgün (uniform) değildir: $P(w)$, kelimenin derlemde bulunma sıklığıdır. Sık geçen kelimeler daha çok, nadir kelimeler daha az seçilir. Böylece sistem, her kelimeyle geçen sık kelimelerden daha çok uzaklaştırmaya çalışır.

Her kelime için k tane negatif örnek alınır. Bağlam 2 iken 4 pozitif ikili gelir; negatif tarafta Mikolov 5 kelime kullanmış ve pratikte iyi sonuç verdiğini görmüştür. Bu işlem pencere kaydıkça bütün kelimeler için tekrarlanır; büyük derlemlerde iyi sonuç verir.

> **Ek bilgi (derste anlatılmadı):** word2vec'in özgün uygulamasında negatif örnekler büyük olasılıkla ham unigram sıklığıyla değil, sıklığın $3/4$ kuvvetiyle orantılı bir dağılımla ($P(w) \propto f(w)^{3/4}$) seçilir. Bu, çok sık kelimelerin seçilme payını biraz azaltıp nadir kelimelerinkini artırır.

6.4 Soru-cevap

Soru: $P(w)$'den seçilen negatif kelimeler, bağlam C'nin içindeki kelimelerden biri de olabilir mi?

Cevap: Evet, mümkün. Frekansı yüksek bir kelimeyse bağlamdaki kelimelerden biri de seçilebilir; o zaman bir yerde onu yaklaştırırken başka bir yerde uzaklaştır demiş oluruz. Bu iyi bir şey değildir. Zaten frekansı yüksek kelimelerin bir sorunu vardır: Jaguar örneğindeki gibi, sık kelimeler genelde çok sayıda anlam barındırır ve her kelimeyle birlikte geçer. Bu da onların koordinatının tek bir sabit vektörle güvenilir biçimde bulunamayacağını gösterir. Bunun çözümü ileride görülecektir.

7. FastText: Kelime İçi n-gram Benzerliği

word2vec 2013 tarihlidir. 2017'de, yazarları arasında yine Mikolov'un da bulunduğu bir ekip FastText'i çıkardı (Bojanowski ve ark., 2017, "Enriching word vectors with subword information").

word2vec kelime tabanlıdır: boşluklarla ayrılmış dizgileri birim (kelime) olarak algılar; sözlük bu birimlerden oluşur. FastText ise kelime altı birimleri (subword) de işe katar. Slayttaki maddeler (ekrandan, 01:10:40):

- Birlikte bulunmaya ek olarak kelime içi n-gram benzerliklerini (sub-word similarity) de dikkate alır.
- Kelimeleri n-gramlarına parçalar ve her bir n-gram için de vektörler bulur.
- Bu sayede sözlükte olmayan (test örneklerinde çok karşılaşılan bir durum) kelimelerin vektörlerini de bulur.
- Türkçe gibi ek alan diller için avantajlıdır (aynı kelimenin farklı ekler almış halleri, yanlış yazılmış halleri birbirine yakın koordinatlara sahip olur).
- Ama daha yavaştır.

Hoca farkı şöyle açıkladı: word2vec iki kelimeyi yalnız birlikte geçiyorlarsa (birinci ya da ikinci dereceden) yakınlaştırır. FastText bunun yanında, içerdikleri benzer karakter n-gramları üzerinden de benzetir.

Ekli diller: “geliyorum” ile “geliyorsun” birbiriyle pek birlikte geçmez. Biraz yakın çıkabilirler, çünkü “geliyorum” ile geçen kelimeler “geliyorsun” ile de geçer; bu ikinci dereceden bir birlikte geçmedir (A ile B birlikte geçmiyor, ama ikisi de C ile geçiyor, bu yüzden yakınlaşıyorlar). Birinci dereceden bir bağ ise yoktur. Birlikte geçmenin yanına karakter n-gram benzerliği konursa, yalnız bir iki karakteri farklı olan bu iki kelime otomatik olarak yakın çıkar.

Yanlış yazımlar: Derlemlerde birçok kelime yanlış yazılır; özel isimlerde, yabancı isimlerde ya da normal yazarken bir harf farklı olabilir. word2vec için bu yepyeni bir kelimedir. FastText ise bir harfi farklı olsa bile bu kelimeleri yakın koordinatlara getirir; ekstra bir şey yapmaya gerek kalmaz. Ancak bu ek işler yüzünden süreç daha yavaştır.

Dağıtım kolaylığı: word2vec ya da FastText ile kelimelerin sabit koordinatları elde edilir. Bir kelimenin koordinatı bir matristen hemen alınır; bu yüzden paylaşması çok kolaydır ve aşırı derecede yaygınlaşmışlardır. FastText yüz küsur dil için bütün kelimelerin koordinatlarını doğrudan yayımlamıştır; Türkçe sözlük 416 bin kelime içerir (ekrandan, 01:20:00).

Dersin sayfasında word2vec’i sıfırdan eğitmek ve eğitilmiş vektörleri kullanmak için MATLAB kodları vardır.

8. Hazır ve Kendi Eğitilmiş Vektörlerle Benzer Kelimeler

Bir kelimeye benzeyen kelimeler şöyle bulunur: kelime modele gönderilir, modelden bir vektör alınır. Bu vektöre (örneğin kosinüs benzerliğiyle) en yakın 20 vektör bulunur ve bu vektörlerin temsil ettiği kelimeler döndürülür. Tablolarda ilk satır sorgu kelimesi, altındaki 19 kelime en yakınlarıdır.

8.1 Hazır FastText vektörleri (n-gram benzerliği var)

Slayttan seçilmiş sütunlar (ekrandan, 01:20:00; “Fasttext vektörleri 416 bin kelime içermektedir. Ngram benzerliği de kullanılır.”):

Sorgu	En yakın kelimelerden bazıları
cimbom	cimbombom, penaltılarla, #beşiktaş, rövanşta, cimbilli, galatasaray, ,fenerbahçe, beşiktaş, kupadaki
terlik	terliksi, terlikler, terlikleri, noterlik, kasiyerlik, önlük, yeterlik, dülgerlik, dizlik, ayakkabıcılık, tonmaysterlik, rençberlik, terli, üzerlik
virüs	virüsün, virüslü, virüsle, virüse, virüstür, virüsü, virüsler, virüsten, mimivirüs, poliovirüs, virüslerle
elma	elma,armut, armut, şeftali, meyveş, meyva, elmaağacı, kayısı, ayva, üzüm, badem
aslan	aslaner, aslana, aslani, aslankeser, aslanyürek, özaslan, sarıaslan, yavuzaslan, aslanlar
koş	koşo, koşoy, koşaç, koşa, koşağaç, koşovi, koşun, koşamaz, koşanlar, koşunca
bırak	bıraktın, bırakın, bırakamam, bırakmam, biraz, bıraksın, bıraktım, bırakayım, bırakırsan

Hocanın yorumları:

- “cimbom” için “cimbombom” ve Galatasaray ile sporla ilgili kelimeler gelmiştir. “cimbilli” birlikte geçmeden değil, “cim” ve “bom/bil” karakter benzerliğinden yakalanmıştır.
- “terlik” daha ilginçtir. “terliksi”, “terlikler” makuldür. Ama “noterlik”? Terlik ile noterlik arasında birlikte geçme üzerinden hiçbir ilişki çıkmaz. Yakın çıkmalarının tek sebebi FastText’in karakter n-gramlarını işe katmasıdır. Bu yakınlığın faydası yoktur, zararı vardır;

"kasiyerlik", "tonmaysterlik", "rençberlik" de öyledir. "önlük" ise birlikte geçmelerden gelebilecek bir kelimedir.
- "virüs" için ise "virüsün", "virüsü", "virüsle" gibi ekli hallerin hepsi yakındır, çünkü ortak karakter n-gram sayıları çok fazladır. Bu istenen bir durumdur. Ancak başına bir şeyler eklenmiş farklı kelimeleri de ("mimivirüs" gibi) yaklaştırır.

8.2 42 bin haber metniyle eğitilen vektörler (n-gram benzerliği yok)

Hocanın grubu 42 bin haber metniyle kendi vektörlerini eğitmiştir; sözlükte 19 bin kelime vardır (ekrandan, 01:21:10):

Sorgu	En yakın kelimelerden bazıları
cimbom	kırmızılılar, beşikta, wesley, didier, galatasaraya, transfere, kırmızılı, sarıkırmızılı, ezeli, futbolcunun
istanbul	sergi, gerçekleşecek, fashion, konser, buluşacak, sergisi, festival, konseri
virüs	virüsü, virüsün, bulaşan, hiv, hastalık, yoluyla, grip, aşı, zararlı, yazılımı, bilgisayar, solunum, enfeksiyon
elma	meyve, meyveler, lif, sebze, limon, çürük, salata, pirinç, buğday, zeytinyağı, patates, muz, fındık
aslan	aslanın, lakaplı, cimbom, kırmızılı, kırmızılılar, devler, didier, drogbayı, lokomotif
ölmek	yaşamak, istemiyorum, asker, erbakan, chavez, lütfen, kanserin, ameliyata, ölümü
bırak	bırakın, dedikleri, kardeşim, git, dediğini, hayattan, yahu

Burada n-gram benzerliği yoktur; hepsi birlikte geçmeye dayanır. Yine de sonuçlar hiç fena değildir. "virüs" örneği eş sesli kelime sorununu da gösterir: hastalıkla ilgili kelimelerin ("hiv", "grip", "aşı") yanında "zararlı", "yazılımı", "bilgisayar" gibi bilgisayar virüsleriyle ilgili kelimeler de listeye girmiştir. "ölmek"e en yakın kelimelerden biri zıt anlamlısı "yaşamak"tır; zıt anlamlı kelimeler gerçekten birlikte geçer. Listede "erbakan", "chavez" gibi kelimeler de vardır; hoca bunların büyük olasılıkla bir haberde birlikte geçtiğini söyledi.

Karşılaştırma: n-gram benzerliğini kullanmasak da iyi sonuçlar alınabilir. Kullanmanın güzel tarafı, özellikle fiillerde farklı ekler almış hallerin hepsini benzer bulabilmesidir. n-gram benzerliği olmadan örneğin "virüsler" kelimesi "virüs"e, listedeki diğer kelimelerden daha uzak kalabilir. Duruma göre bu kabul edilir ya da FastText'e geçilir.

9. Kelime Vektörlerini Değerlendirmek

Kelime vektörlerinin başarısını ölçmenin iki yolu vardır; bu, dil modellerinin değerlendirilmesine benzer:

- **Dahili yöntemler:** İnsanların verdiği cevaplarla korelasyon kullanılır. Benzerlik ve analoji kümeleri vardır; insanlar bunları bir şekilde etiketlemiştir.
- **Harici yöntemler:** Kelime vektörleri başka uygulamalarda kullanılır ve o uygulamalardaki performansa bakılır: duygu analizi (sentiment analysis), metin sınıflandırma gibi.

10. Cümle ve Metin Vektörleri

word2vec ya da FastText ile kelimelerin koordinatlarını bulduk. Ama üzerinde çalıştığımız birimler genelde yalnız kelime değildir; cümlelerle, paragraflarla uğraşırız. Bir cümlenin temsilini nasıl buluruz?

Slayttaki maddeler (ekrandan, 01:25:20):

- En basit yol: içindeki kelimelerin ortalaması.
- Universal sentence encoder (https://arxiv.org/abs/1803.11175).

Slayttaki şekilde “How old are you?”, “What is your age?”, “My phone is good.” gibi cümleler bir “Embed” kutusuna verilir ve her biri için bir vektör çıkar ([0.3, 0.2, …], [0.2, 0.1, …], [0.9, 0.6, …]). Yanındaki “Semantic Textual Similarity” ısı haritasında cümleler arasındaki benzerlik (aralarındaki açı üzerinden) gösterilir. “How old are you?” ile “What is your age?” birbirine çok benzer çıkar; “My phone is good.” ile “Your cellphone looks great.” de birbirine benzer çıkar. “What is your age?” ile “My phone is good.” arasında ise benzerlik düşüktür.

Hoca, bu gösterimi anlatırken kelime vektörlerinin bilindiğini varsayıp yapılan tek işlemin ortalama almak olduğunu düşünebileceğimizi söyledi.

> **Ek bilgi (derste anlatılmadı):** Universal Sentence Encoder (Cer ve ark., 2018) büyük olasılıkla iki türle yayımlanmıştır: kelime ve bigram gömmelerinin ortalamasını derin bir ileri beslemeli ağdan geçiren DAN (Deep Averaging Network) ve transformer tabanlı bir kodlayıcı. DAN türü, ortalama alma fikrinin üzerine öğrenilmiş katmanlar ekler.

Aradan sonra hoca, metin koordinatını kelime koordinatlarının ortalamasıyla bulma fikrinin word2vec’te de BERT’te de çok farklı olmadığını belirtti: bu modellerde kelimelerin koordinatlarının elde edilme süreci farklıdır, ama bir metnin koordinatının elde edilmesi buna çok benzer.

11. Kelime Öbekleri

11.1 Tanım

İyi bir kelime öbeği, parçaları birleştiğinde parçalarından farklı bir anlam ifade eden yapıdır. “Fatih Sultan Mehmet” üçlü bir kelime öbeğidir; “Fatih”, “Sultan” ve “Mehmet”in her birinden farklı bir kavramı içerir.

“Fatih Sultan Mehmet” ve “Fatih Terim” ifadelerini düşünün. Bunları kelime öbeği olarak almazsanız ikisinde de “Fatih” geçtiği için ortak bir yanları varmış gibi düşünülebilir. Kelime öbeği olarak alırsanız her biri ayrı bir birim olur ve aralarında ortaklık kalmaz. Yani kelime öbekleri sözlüğün içine katılabilir.

11.2 Kelime n-gramları

Kelime n-gramı, karakter n-gramından farklıdır: karakter n-gramı art arda gelen karakterlerden, kelime n-gramı art arda gelen kelimelerden oluşur. Bir uzunluğundaki kelime n-gramı kelimenin kendisidir; iki uzunluğunda A’dan sonra B kelimesi, üç uzunluğunda A, B, C gibi yapılar olur.

En basit yöntem frekansı en yüksek olanları denemektir. 41.992 haberlik derlemde nadir olanlar silinmeden n-gram sayısı 11 milyona çıkmış; nadirler silinince 8 milyon civarına inmiştir. Frekansı en yüksek n-gramlar (ekrandan, 01:40:00):

Uzunluk	En sık n-gramlardan bazıları (sayı)
1	“ve” (184.540), “bir” (149.920), tırnak işareti (121.610), “bu” (93.130), “da” (73.033), “de” (70.376), “için” (57.285), “ile” (54.426), “çok” (39.307), “olarak” (34.066)
2	tırnak + “nin” (9.411), tırnak + “ın” (9.061), tırnak + “de” (8.796), “türkiye” + tırnak (6.541), “diye konuştu” (6.450), tırnak + “dedi” (6.240), “ya da” (5.226)
3	“türkiye” + tırnak + “nin” (2.187), “türkiye” + tırnak + “de” (2.171), tırnak + “diye konuştu” (2.168), “spor toto süper” (1.580), “şöyle devam etti” (1.509), “başbakan recep tayyip” (1.407)

Tekliler için slayttaki değerler 1.8454e+05, 1.4992e+05 ve 1.2161e+05 biçiminde yazılıdır. Hoca, slayttaki kutu karakterini "tırnak işareti" olarak okudu; Türkçe özel isimlere gelen ekleri ayıran kesme işareti olduğu büyük olasılıktır.

11.3 Sık n-gramları tek kelimeye çevirmek

Slayttaki adımlar (ekrandan, 01:41:20):

- 8 milyon n-gramdan sadece 36 bin'i 50'den fazla kez geçiyor. Sadece bunlarla ilerlenir; tekil kelime sayısı 20 bin olur.
- n-gramlar, kendi içlerindeki kelimelerden ayrı kendi başlarına anlamları / koordinatları olsun diye tek kelimelere çevrilir.

Örnek dönüşümler:

```
"bitirdikten sonra"    -> "bitirdiktensonra"
"yer alırken"          -> "yeralırken"
"700 milyon"           -> "700milyon"
"sırada yer alırken"   -> "sıradayeralırken"
"kız kardeşi"          -> "kızkardeşi"
"cep telefonundan"     -> "ceptelefonundan"
"internet üzerinden"   -> "internetüzerinden"
"başbakan da"          -> "başbakanda"
"ve şöyle"             -> "veşöyle"
"bir tane"             -> "birtane"
```

Bu yapılınca haber metinlerinde birden fazla kelime içeren yeni "kelimeler" oluşur: "Dışişleri Bakanı", "görüşmelerin ardından", "cenaze töreni", "hastalığı nedeniyle" gibi. Hoca "sanatseverler"in ayrı mı bitişik mi yazıldığından emin olmadığını, "sarı kırmızı"nın ise ayrı yazıldığını söyledi. Böylece kelime öbeklerini de işe katmak mümkündür. Hoca slaytlarda verilen kaynakları incelemeyi önerdi.

> **Ek bilgi (derste anlatılmadı):** Mikolov ve ark. (2013) word2vec makalesinde de benzer bir yöntem büyük olasılıkla kullanılmıştır: iki kelimenin birlikte geçme sayısı, ayrı ayrı geçme sayılarının çarpımına göre yüksekse ikili tek bir belirteç (örneğin "New_York") yapılır; işlem birkaç kez tekrarlanarak daha uzun öbekler de bulunur.

12. Bölüm 4'e Giriş: Makine Öğrenmesinin İkinci Versiyonu

Bölüm 3 bitince, bir sonraki hafta için Bölüm 4'e başlandı. Hoca bu bölüme "makine öğrenmesinin ikinci versiyonu" (ML v2) adını verdi. Birinci versiyonda belli türde temel problemler vardı; ikinci versiyonda yeni tür problemler ve teknikler vardır. Bölüm içeriği oldukça yoğundur ve yeni şeyler eklendikçe büyümektedir (in-context örnekler gibi).

Hoca bu slaytların doğrudan hesaplamalı anlambilime özel olmadığını, kolektif öğrenme dersini alanların bunları orada da gördüğünü söyledi. Amaç, makine öğrenmesinde günümüzde neler olduğu, hangi alanların araştırma konusu olarak sıcak olduğu ve hangi tekniklerin bulunduğu konusunda vizyon kazandırmaktır. Bazıları hesaplamalı anlambilimle çok ilgilidir, bazıları değildir; ama tezlerde kullanılabilirler.

13. Aktif Öğrenme

13.1 Tanım ve motivasyon

Aktif öğrenme (active learning), eğitim örneklerinin etiketlerinin başta belli olmadığı, algoritmanın etiketini istediği örnekleri seçtiği bir öğrenme biçimidir. Normalde elimizde bir eğitim kümesi vardır; örneklerin hem içeriğini hem etiketini biliriz ve buna göre bir model kurarız. Aktif öğrenmede ise örnekler vardır ama etiketleri yoktur. Gerçek dünyada etiketler belli bir para verilerek elde edilir. Dolayısıyla aktif öğrenme, veri etiketleme sürecinin maliyetini optimize etmekle ilgilidir.

Fikir: rastgele seçilmiş 500 veriyi etiketletmek yerine akıllıca seçilmiş 500 veriyi etiketletmek daha iyi olabilir. Elimizde 1000 ya da 10.000 veri ve 500 veri etiketletecek kadar para olsun. Hangi 500'ü insanlara etiketletelim? Rastgele seçimden daha iyi bir şey yapılabilir mi?

Örneğin 1000 örneğin hepsi etiketlenince sağlam bir başarıya erişiliyor, ama rastgele 500 tanesi etiketlenince o kadar iyi bir başarıya erişilemiyor olsun. Amaç, rastgele 500'den daha yüksek, ama büyük olasılıkla bütün 1000'den daha düşük bir başarıyı daha az etiketle elde etmektir. Tüm eğitim setini kullanmadan, belirli bir test seti üzerinde kabul edilebilir derecede düşük, belki de daha yüksek bir başarı hedeflenir; ama daha yükseği pek beklenmez.

13.2 Adımlar

Bu işlem genelde tek adımda ilerlemez; bir seferde "şu verilerin etiketlerini istiyorum" diye karar verilmez, adım adım ilerlenir. Slayttaki adımlar (ekrandan, 01:50:00):

- **A:** Küçük bir eğitim setiyle (E) başla (eğitim kümesinin az bir kısmının etiketlerini iste).
- **B:** Bir model üret. Bu modele göre etiketsiz eğitim örneklerine (EE) dair tahminlerini üret.
- **C:** Bu tahminlere göre bu örneklerden bir kısmını seç (tahmin varyasyonuna göre) ve etiketlerini iste (EE'den çıkar, E'ye ekle).
- B ve C adımlarını tekrar et.

Hoca bunu bir tabloyla adım adım anlattı (ekrandan, 02:00:00):

Etiketli	Etiketsiz
0	1000
100 (M1)	900 (M1'e göre tahminler)
200 (M1)	800 (M1'e göre tahminler)
…	…
1000	0

1. Başta hiçbir etiket yoktur. 1000 örnek içinden rastgele 100'ü seçilir ve etiketleri istenir. Artık bu 100 örneğin hem girişi hem çıkışı bilinir.
2. Bu 100 örnekle bir model (M1) eğitilir.
3. M1 kalan 900 örneğe verilir ve tahminler üretilir. Bu 900'ün etiketleri bilinmediği için hangilerinin doğru, hangilerinin yanlış tahmin edildiği bilinemez. Hoca bütçe benzetmesi yaptı: 500 liramız vardı, 100'ünü harcadık, 400 lira kaldı.
4. Tahminlere göre 900 içinden 100 örnek daha seçilir ve etiketleri istenir. Artık 200 etiketli örnek vardır.
5. 200 örnekle model yeniden eğitilir, kalan 800 için tahmin alınır, yine 100 seçilir ve süreç böyle devam eder.

Sınıfa sürecin sonunda ne olacağı soruldu. Cevap: devam edilirse bütün veri etiketlenmiş, etiketsiz veri sıfıra inmiş olur. Ama asıl hikâye oraya varmadan, örneğin 200'lük bir bütçeyle yeterince iyi bir şey yapılıp yapılamayacağıdır; son satır zaten paramızın yetmeyeceği yerdir.

13.3 Hangi örnek seçilir?

Hoca şu soruyu sordu (ekrandan, 02:00:00): veri kümesi iki sınıflı (A ve B) olsun. M1'e göre:

- Örnek 1: A: 0.51, B: 0.49
- Örnek 2: A: 0.95, B: 0.05

Bu iki örnekten hangisinin gerçek etiketini öğrenmek istersiniz? Bir öğrencinin cevabı: olasılıkları birbirine yakın olanın. Hoca onayladı: Örnek 2'nin etiketi büyük olasılıkla A'dır (kesin değil, yanlış da olabilir), ama Örnek 1'de modele hiç güvenilemez. Bu yüzden Örnek 1'i öğrenmek daha makuldür. 900 örnek içinden sınıf olasılıkları birbirine yakın olan 100 örnek seçilir.

Bir gösterimde (dersin sayfasında kodları olan bir deney) üç sınıflı bir veri kümesi kullanılmıştı: sarı, kırmızı ve mavi. Üçgenler etiketi bilinen mevcut eğitim örnekleri, yuvarlaklar bir iterasyonda seçilen örneklerdir. Seçilen örnekler kırmızı ve sarı sınıfların sınır bölgesinde yoğunlaşmıştır; mavi sınıf çok uzakta olduğu için onunla hiç uğraşılmamıştır. Uzaktaki örneklerin etiketi aşağı yukarı bellidir, onlar için para harcamaya gerek yoktur.

Hocanın vurgusu: Asıl önemli olan sınır bölgeleri, işlerin birbirine girdiği yerlerdir. Model oralarda emin değildir; oradan öğrendikten sonra süreç yeniden aynı şekilde devam eder.

13.4 Açık sorular

Slayttaki sorular üzerinden şunlar tartışıldı:

- **Başlangıç seti nasıl seçilmeli?** Rastgele mi, yoksa önce kümeleme yapıp kümelerden mi? Bunun önemi vardır, çünkü bütün süreç başlangıca bağlı olarak ilerler.
- **Tahmin varyasyonu nasıl hesaplanır?** Sınıflandırmada sınıf olasılıklarının birbirine yakınlığına bakılabilir. Regresyonda da belli varyanslar hesaplanabilir ya da çıkışta büyük geçişlerin olduğu bölgelerden örnek istenebilir.
- **B ve C ne zamana kadar tekrarlanır?** Büyük olasılıkla bütçe yetene kadar.
- **C'de kaç örnek eklenir?** Örnekte 100'den başlanıp 100'er ilerlendi; bu bir hiperparametredir ve farklı yapılarda kullanılabilir.

13.5 Deneyler

Önce kolay örnekler (ekrandan, 02:02:40): 6 veri kümesinde (segment, vehicle, autos, iris, hillValley, audiology) iki yöntem karşılaştırılmıştır: rastgele ekleme ve "önce kolayları veren" aktif öğrenme. Kolay örnekler, %95'e %5 gibi etiketi büyük olasılıkla doğru tahmin edilen örneklerdir. X ekseni adımları (her adımda bir parça eklenir), Y ekseni test kümesi üzerindeki başarıyı gösterir. Veri 20 parçaya bölünmüştür; 20 adımda bütün verilerin etiketine erişilir. Bu yüzden iki yöntemin son noktası aynıdır. Örneğin 10. adım, eğitim kümesinin yarısının kullanıldığı yerdir: rastgele yöntemde yarısı rastgele, aktif öğrenmede yarısı adım adım (100, 100, 100… kümülatif olarak) seçilmiştir.

Sonuç: rastgele ekleme, önce kolayları vermeye göre genelde daha iyidir (segment'te bariz biçimde). Bazı veri kümelerinde arada büyük fark yoktur. Yani aktif öğrenmeyi bu şekilde kullanmanın pek anlamı yoktur.

Soru: Y eksenindeki rakamlar ne oluyor?

Cevap: O veri kümesinin test kümesi üzerindeki başarısını gösteriyor.

Önce zor örnekler: Zor örnekler önce verildiğinde aktif öğrenme bir miktar daha işe yaramıştır. Bir veri kümesinde eğitim kümesinin yaklaşık yarısı kullanıldıktan sonra daha fazla etiketli veri eklemek pek bir şey değiştirmemiş, rastgele yöntemde ise artış devam etmiştir. Hoca bunu aktif öğrenmenin başarılı olduğu bir deney olarak yorumladı: verinin yarısıyla bütününe yakın başarıya ulaşılmıştır. Bu durum her veri kümesinde yoktur; bazılarında iki yöntem arasında pek fark yoktur.

İlginç bir durum daha görüldü: bir veri kümesinde tüm veriyle %54 civarında başarı elde edilirken verinin bir kısmıyla %60'ın üstüne çıkılabilmiştir. Bu, veri kümesindeki bazı örneklerin büyük olasılıkla gürültülü olduğunu ve dışarıda bırakılınca daha iyi sonuç alındığını gösterir. Örnek eklendikçe başarının kötüleştiği durumlar da veri kümesinin sorunlu olduğuna işaret eder.

13.6 Bulgular

Slayttaki özet (ekrandan, 02:09:05):

- Önce kolay örnekleri eklemek iyileştirmiyor.
- Önce zor örnekleri eklemenin yararlı olduğu veri kümeleri var.
- Bazı veri kümelerinde tüm eğitim örnekleri yerine daha az örnekle daha yüksek test başarılarına erişilebiliyor.

Daha fazlası için slaytta "Human-in-the-Loop Machine Learning" (Manning) kitabı önerilmiştir.

14. Yarı Eğiticili Öğrenme

14.1 Tanım ve temel fikir

Slayttaki tanım (ekrandan, 02:10:00):

- Modelin oluşturulmasında etiketli verilerin yanında etiketsiz verilerin de kullanıldığı problemlerdir.
- Temel fikir: Elde hem etiketli hem de etiketsiz veriler var. Belirli bir test seti üzerinde, sadece etiketli verileri kullanmak yerine etiketsizleri de kullanmak daha iyi olur.

Slayttaki şekilde, abartılı bir örnek olarak, yalnız iki etiketli nokta (biri içi boş, biri dolu) vardır. Yalnız bunlar kullanılırsa aralarından düz, dikey bir karar sınırı çekilir. Etiketsiz (gri) noktalar da dikkate alınınca, bu noktaların oluşturduğu iki kıvrımlı grubun arasından geçen eğri bir karar sınırı çekmek daha doğru olur.

Aktif öğrenmeden farkı: burada etiket satın alacak bir bütçe yoktur. Elde etiketli veriler ve etiketi bilinmeyen veriler vardır; etiketsizleri bir insana etiketletme şansı yoktur.

14.2 Etiketsizlerin sınıfını tahmin etmek

Çözüm, etiketsiz örneklerin sınıflarını tahmin etmektir. Gri noktaların hepsinin hangi renkten olduğunu bilsek ya da doğru tahmin etsek, eğri sınırı rahatlıkla çizebiliriz. Bunun için bir tahmine ihtiyaç vardır; tahmin doğru da yanlış da çıkabilir. Bu yüzden belki iteratif bir şekilde, tekrar tekrar yapmak gerekir.

Tahmin nasıl yapılır? Etiketli verilerle bir model kurulur ve bu model etiketsizlerin etiketlerini tahmin etmek için kullanılır. Tahmin edilen etiketlerin hepsini kullanmak yerine bir kısmını kullanmak daha iyi olabilir; aktif öğrenmedeki gibi yüzer yüzer ilerlemek makul bir yaklaşımdır. Regresyon problemlerinde ise tahminlerin güvenilirliği için örneğin ortalamadan uzaklığa bakmak gibi yaklaşımlar düşünülebilir.

14.3 Tek adım mı, iteratif mi?

Hoca slayttaki şekil üzerinden sınıfa sordu: bu veri kümesinde etiketleme tek adımda mı yapılmalı, iteratif mi?

Soru (hocadan): Tek adım mı, iteratif mi?

Cevap (öğrenciden): İteratif daha iyi gibi; sınıf etiketleri her iterasyonda değişebilir.

Hoca cevabı doğru buldu ve gerekçeyi açtı. Elde yalnız iki etiketli nokta varken kurulan modelin karar sınırı dikey bir çizgidir. Bu modelle bütün etiketsizler tek adımda etiketlenirse çizginin bir tarafındakilerin hepsine bir sınıf (örneğin kırmızı), öbür tarafındakilerin hepsine diğer sınıf (sarı) denir. Bir sonraki adımda model bu etiketlerle yeniden eğitildiğinde karar sınırı pek değişmez, çünkü etiketler zaten o çizgiye göre verilmiştir; tahminler de değişmez.

Daha dinamik bir süreç için adım adım ilerlenir: her adımda yalnız en güvenilen örnekler (etiketli noktaya çok yakın olanlar, “konfor alanı” içindekiler) etiketlenir. Bir sonraki adımda bu yeni etiketlerden yola çıkılarak biraz daha uzaktakiler etiketlenir. Böylece etiketler kıvrımlı grupların içinden ilerleyebilir.

14.4 Hata birikimi

Etiketsizlerin hepsini tahmin etmek de soru işaretlidir. İteratif bir süreçte her adım bir öncekine bağlıdır; önceki adımlardaki hatalar ileriye doğru kümülatif olarak taşınır. Başlangıçta etiketli çekirdeğe yakın örnekler için tahmin yapılırken zamanla daha uzak örnekler için, yani daha az güvenilen tahminler yapılmaya başlanır. Bu yüzden etiketsiz verinin tamamını kullanmak iyi bir hareket olmayabilir; bir kısmı işimize daha çok yarayabilir. Her adımda kaç örnek ekleneceği (adım büyüklüğü) de bir sorudur.

14.5 Deneyler

Slayttaki grafikler (ekrandan, 02:20:00) 6 veri kümesinde (segment, vehicle, autos, iris, hillValley, audiology) 10 iterasyon boyunca test başarısını gösterir:

- **Supervised (mavi):** yalnız etiketli veriyle kurulan model; düz bir çizgi.
- **Batch semi-supervised (kırmızı):** etiketsizlerin hepsi tek adımda tahmin edilip eklenir; düz bir çizgi.
- **Iterative semi-supervised (sarı):** adım adım ekleme; iterasyonlarla değişen bir eğri.

Hoca, buradaki "supervised"ın bütün verinin etiketlerinin bilindiği durum değil, yalnız elimizdeki etiketli verilerin kullanıldığı durum olduğunu vurguladı. Amaç bütün veri etiketliyken elde edilecek başarıyı aşmak değil, yalnız etiketli veri kullanılan durumu aşmaktır.

Grafiklerden okunanlar (değerler grafikten yaklaşık):

- **segment:** supervised ve batch yaklaşık 0.70; iteratif 0.66–0.69 arasında, aşamamış.
- **vehicle:** supervised yaklaşık 0.484, batch 0.459; iteratif 0.443–0.483 arasında dalgalanmış, aşamamış.
- **autos:** supervised yaklaşık 0.371, batch 0.378; iteratif 5. iterasyonda 0.42'nin üstüne çıkmış, sonra düşmüş.
- **iris:** üst çizgi yaklaşık 0.929; iteratif 0.857–0.92 arasında, aşamamış.
- **hillValley:** supervised 0.50, batch yaklaşık 0.489; iteratif ilk iterasyonda yaklaşık 0.52 ile aşmış, sonra altına inmiş.
- **audiology:** supervised yaklaşık 0.294, batch 0.31; iteratif 0.38'e çıkarak aşmış.

Yorumlar: bazı veri kümelerinde batch, bazılarında iteratif daha iyidir. İteratif yöntem bazen başta yüksekken zamanla düşmüştür; erken durmak daha iyi olabilir. Birçok veri kümesinde etiketsizlerin hepsini kullanmak yerine bir kısmını kullanmak daha iyi olmuştur. Etiketsizleri hiç kullanmamanın, tek seferde tahmin etmenin ya da iteratif tahminin diğerlerinden iyi olduğu durumlar vardır; her zaman şu daha iyidir demek zordur. İteratif tahminin nerede durdurulacağı sonucu çok etkiler.

> **Hocanın vurgusu:** Sonuçları birden fazla veri kümesinde göstermenin sebebi şudur: tek bir veri kümesine bakarak karar vermek iyi olmayabilir. Yeterince büyük bir veri kümeniz yoksa oynaklık çok fazladır; deneyleri on kez tekrarlamak bile yetmeyebilir. Birkaç farklı veri kümesinde çalışmak sonuçlarınızı daha rahat eleştirmenizi sağlar. Sonuçlarınızı eleştirmek iyidir.

Derin öğrenmede de yarı eğiticili yöntemler vardır; bunlara ileride bakılacaktır.

Örnek Sorular

Soru 1 (ekrandan, 00:10:00): Ali $= [0.7,\ -0.2,\ \ldots,\ 0.8]$ ve Yemek $= [0.5,\ 0.9,\ \ldots,\ 0.7]$ 300 boyutlu kelime vektörleridir. "Ali yemek" metninin vektörü ortalama alma yöntemiyle nedir? "yemek Ali" metninin vektörü ne olur?

> **Çözüm:** Her boyutta ortalama alınır: $(0.7 + 0.5)/2 = 0.6$, $(-0.2 + 0.9)/2 = 0.35$, ..., $(0.8 + 0.7)/2 = 0.75$. Sonuç $[0.6,\ 0.35,\ \ldots,\ 0.75]$'tir ve yine 300 boyutludur. Ortalama sıralamaya bakmadığı için "yemek Ali" de aynı vektörü verir (derste sınıf da bu cevabı verdi).

Soru 2 (ekrandan, 00:30:00): Kelime yörüngeleri yönteminde, farklı uzunluktaki metinler nasıl aynı boyutlu bir vektörle temsil edilir ve bir metin için kaç özellik çıkar?

> **Çözüm:** Her ölçüm türü için (örneğin d uzunluğundaki yörüngedeki $d-1$ adet $(n, n+1)$ mesafesi) eşit aralıklı 10 parçalık histogram çıkarılır; histogramın 10 değeri ve 10 frekansı 20 özellik verir. Bu, $(n, n+1)$, $(n, n+2)$, $(n, n+3)$ mesafelerine ve $(n, n+1, n+2)$, $(n, n+2, n+3)$ açılarına uygulanır: $5 \times 20 = 100$ özellik. Histogram kelime sayısından bağımsız olduğu için boyut sabittir.

Soru 3 (ekrandan, 01:10:00): "w1 w2 w3 w4 w5 w6 w7 ..." metninde $C = 2$ için, w_3 merkez kelimeyken skip-gram'ın benzer ve farklı olmasını istediği ikilileri yazınız. C-bow'da ne değişir?

Çözüm: Benzer olması istenen ikililer (w_3, w_1), (w_3, w_2), (w_3, w_4), (w_3, w_5)'tir. Farklı olması istenen ikililer $(w_3, t_1), \ldots, (w_3, t_K)$'dır; $t_1 \ldots t_K$ $p(w_i)$'ye göre rastgele seçilir. Sonra w_4 ile devam edilir: (w_4, w_2), (w_4, w_3), (w_4, w_5), (w_4, w_6). C-bow için ikililer ters çevrilir (bağlam kelimesi giriş, merkez kelime çıkış olur).

Soru 4 (derste sözlü, 01:08:27): Negatif örnekleme için $P(w)$'den seçilen bir kelime bağlam C içindeki kelimelerden biri olabilir mi?

Çözüm: Hocanın cevabı: evet, özellikle frekansı yüksek kelimeler için mümkündür; bu iyi bir şey değildir, çünkü kelime bir yerde yaklaştırılırken başka yerde uzaklaştırılır. Sık kelimeler zaten çok anlamlı olduklarından koordinatları tek bir sabit vektörle güvenilir bulunamaz.

Soru 5 (ekrandan, 02:00:00): İki sınıflı bir problemde aktif öğrenme yapılıyor. M1 modeline göre Örnek 1 için A: 0.51, B: 0.49; Örnek 2 için A: 0.95, B: 0.05 tahmini var. Hangi örneğin etiketini istemek daha makuldür?

Çözüm: Örnek 1. Modelin iki sınıfa verdiği olasılıklar birbirine çok yakın olduğu için model bu örnekte emin değildir; örnek büyük olasılıkla sınıfların sınır bölgesindedir. Örnek 2'nin etiketi büyük olasılıkla A'dır, onun için para harcamak gereksizdir (derste sınıf da bu cevabı verdi).

Soru 6 (derste sözlü, 01:57:01): Aktif öğrenmede her adımda 100 örnek etiketletilerek 1000 örneklik kümede B ve C adımları tekrarlanırsa süreç nereye varır? Aktif öğrenmenin asıl amacı nedir?

Çözüm: Sonunda bütün veri etiketlenir (1000 etiketli, 0 etiketsiz). Asıl amaç oraya varmadan, örneğin 200'lük bir bütçeyle, bütün verinin etiketlenmesiyle erişilecek başarıya yakın bir başarı elde etmektir.

Soru 7 (derste sözlü, 02:13:53): Yarı eğiticili öğrenmede yalnız iki etiketli nokta ve kıvrımlı iki grup oluşturan çok sayıda etiketsiz nokta varken, etiketsizlerin hepsini tek adımda tahmin edip eklemek mi, iteratif ilerlemek mi daha iyidir? Neden?

Çözüm: İteratif ilerlemek daha iyidir. İki noktadan kurulan modelin karar sınırı düz bir çizgidir; tek adımda bütün etiketsizler bu çizgiye göre etiketlenirse yeniden eğitimde sınır değişmez. İteratif yöntemde her adımda yalnız en güvenilen (etiketli noktaya yakın) örnekler etiketlenir ve etiketler kıvrımlı grupların içinden yayılabilir. Ancak hatalar adımlar boyunca birikebileceği için etiketsizlerin hepsini kullanmak her zaman iyi değildir.

Dersin Toparlaması

Ders, kelime koordinatlarından metin koordinatına ortalama alarak geçmekle başladı. Bu basit yöntem, sırayı yok saymasına rağmen haber sınıflandırmada 4500 boyutlu seyrek temsilden daha iyi sonuç verdi. Ancak birlikte geçme matrisine dayanan yöntemlerin $V \times V$ yapısı ölçeklenmez ve tek vektörlü temsil çok anlamlı kelimelerde sorun çıkarır. Kelime yörüngeleri çalışması ise kelime koordinatlarının içeriğine bakmadan, yörüngenin mesafe ve açı histogramlarıyla yazar tanımanın mümkün olduğunu gösterdi.

Ardından sinirsel dil modellerine geçildi. word2vec'in skip-gram ve C-bow yapıları, pencereyle üretilen eğitim ikilileri, iç çarpıma dayalı hata fonksiyonu ve uzayın çökmesini önleyen negatif örnekleme anlatıldı. word2vec'in asıl kazancı $V \times V$ matrise ihtiyaç duymamasıdır. FastText kelime içi n-gram benzerliğini ekleyerek eklemeli dillerde ve yanlış yazımlarda avantaj sağlar, ama ilgisiz kelimeleri de yakınlaştırabilir. Kelime öbekleri, sık kelime n-gramlarının tek kelime yapılmasıyla sözlüğe katılabilir. Cümle vektörleri için en basit yol yine ortalamadır.

Son kısımda Bölüm 4'e ("makine öğrenmesinin ikinci versiyonu") başlandı. Aktif öğrenme, etiketleme bütçesini modelin emin olmadığı sınır bölgelerine harcamayı; yarı eğiticili öğrenme ise etiketsiz verilerin tahmini etiketlerini de kullanmayı amaçlar. Deneyler iki yöntemin de her veri kümesinde işe yaramadığını, sonuçları birden çok veri kümesinde sınamanın önemini gösterdi. Bölüm 4'ün geri kalanına sonraki derste devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.